Etapa 3: Infraestrutura, Segurança e Documentação

Disciplina: Big Data e Analytics, UCSAL 2026.2
Alunos: Caio Oliveira Sampaio e Victor Oliveira Santana Silva
Tema: Diversidade na Tecnologia: Raça, Gênero e o Funil de Liderança no Setor de TI Brasileiro
Repositório: github.com/the-Victorr/ucsal-bigdata-diversidade-ti

Nas etapas anteriores a gente tratou os dados e carregou no banco. Agora o assunto é como tudo isso está montado: a infraestrutura escrita em código, as senhas fora do repositório, os arquivos organizados e o README explicando o que foi feito.

1. Infraestrutura como código

A infraestrutura do banco está escrita em arquivos Terraform, que é a ferramenta que descreve servidor em código. Quem baixar o repositório consegue criar o mesmo banco na conta dele sem clicar em nada no painel do Aiven.

```
terraform/main.tf                    recursos aiven_pg e aiven_pg_database
terraform/variables.tf               parametros, com o token marcado como sensivel
terraform/outputs.tf                 host, porta e string de conexao
terraform/terraform.tfvars.example   modelo de preenchimento, sem valores
```

São dois recursos declarados. Um é o servidor PostgreSQL, com o plano, a região e a versão. O outro é o banco que fica dentro dele. O Aiven já cria um banco chamado defaultdb, que foi o que usamos na carga, e o segundo recurso deixa escrito o banco próprio do projeto.

O que está provisionado

Recurso	Configuração
Serviço	ti-diversidade-pg
Projeto no Aiven	ucsal-bigdata-2026
Plano	free-1-5gb, com 1 GB de armazenamento
Nuvem e região	DigitalOcean, sfo
Versão	PostgreSQL 18
Acesso	internet pública, com TLS obrigatório
Bancos lógicos	defaultdb e diversidade_ti

Uma coisa que vale contar: criamos o serviço pelo painel enquanto desenvolvíamos, e só depois escrevemos o Terraform copiando essa configuração. No primeiro rascunho o código apontava pra outra nuvem e outra versão do PostgreSQL. Fomos conferir no painel e corrigimos, senão o código estaria descrevendo um servidor diferente do que está rodando de verdade.

O plano gratuito do Aiven desliga o servidor sozinho depois de um tempo sem ninguém conectar. Os dados ficam guardados e ele volta pelo botão de ligar no painel, mas é bom saber disso antes de tentar acessar e achar que a senha está errada.

2. Segurança das informações

Nenhuma senha, token ou chave está no repositório público. Cada uma tem seu lugar.

Credenciais do banco

Ficam num arquivo .env na raiz do projeto, que está no .gitignore e nunca subiu pro GitHub. O repositório leva só o .env.example, que mostra quais variáveis existem mas com os valores vazios. O script da carga lê isso e, se faltar alguma, ele para e diz qual é, em vez de tentar conectar com um valor qualquer.

Token da API do Aiven

Só o Terraform usa. Ele vem de variável de ambiente e está marcado como sensitive = true, que faz o valor não aparecer na tela quando roda o comando. O arquivo terraform.tfvars, que seria onde esse token ficaria escrito, também está no .gitignore. O repositório leva só o exemplo.

Estado do Terraform

Os arquivos .tfstate guardam as senhas do servidor em texto puro, então também ficam fora do repositório. Isso é fácil de esquecer em projeto com Terraform, por isso deixamos registrado que pensamos nisso.

Dados pessoais

Usamos a versão pública dos microdados, que já vem sem CPF, PIS, nome e data de nascimento. Mesmo assim o ETL joga fora as colunas de bairro e distrito, porque bairro junto com profissão e faixa de salário chega perto de identificar alguém. No banco, as tabelas com o dado completo ficam num esquema interno, e as públicas escondem qualquer combinação com menos de 5 pessoas.

Separação de privilégio

O banco tem um usuário chamado dashboard_leitura, que só enxerga as tabelas públicas e não muda nada. Criamos ele sem senha de propósito, assim nenhuma credencial fica escrita no repositório. A senha é definida na hora que alguém precisa do acesso.

3. Repositório

O repositório está em github.com/the-Victorr/ucsal-bigdata-diversidade-ti, público, com 31 arquivos e 13 commits. Cada pasta guarda um tipo de coisa:

```
etl/        scripts do pipeline, do download ate a carga
sql/        estrutura do banco e consultas de exemplo
terraform/  infraestrutura como codigo
docs/       dicionario de dados
```

```
data/       base tratada em CSV e em Parquet
README.md   documentacao do projeto
```

Separamos os commits por etapa do pipeline pra dar pra acompanhar a sequência: primeiro o catálogo das fontes e o download, depois a extração com filtro, a limpeza, o modelo do banco, a carga, o Terraform e por último a documentação.

Os arquivos baixados do governo ficam em data/raw e data/processed, fora do repositório. São 4,2 GB compactados e mais de 40 GB abertos, não daria pra subir isso. Já a base tratada está no repositório em CSV e em Parquet, porque é o resultado do trabalho e cabe em 15 MB.

4. Documentação

Reescrevemos o README pra cobrir os cinco blocos pedidos.

Bloco	O que tem lá
Dados utilizados	fonte no FTP do PDET, identificação das duas bases, período de 2025 e as características que pesaram nas decisões
Tratamento realizado	os problemas encontrados na base, as decisões tomadas e a justificativa de cada uma, numa tabela
Modelagem	estrutura do banco, por que modelo estrela, o grão escolhido e onde está o dicionário de dados
Infraestrutura	serviços, recursos provisionados e as tecnologias usadas
Execução	dependências, o que precisa estar preenchido e a sequência de comandos

Fora o README, tem o dicionário de dados com as 131 colunas. Ele não foi escrito à mão: sai de uma consulta que lê os comentários do próprio banco. Assim ele não fica desatualizado, porque se a coluna mudar e o comentário for corrigido, o dicionário sai certo da próxima vez que a gente gerar.

5. Escolhas técnicas e por que elas

Escolha	Alternativa descartada	Motivo
Pandas com leitura em blocos	PySpark	O volume depois do filtro cabe em memória. Subir um cluster só para ler CSV seria complexidade sem retorno
Parquet como formato intermediário	CSV	Comprime melhor e guarda o tipo da coluna, o que evita reconverter a cada passo
PostgreSQL relacional	Banco de documentos	O dado é tabular, com esquema fixo, e as consultas são agregações com filtro cruzado
Modelo estrela	Modelo normalizado	Cada filtro do painel vira uma junção simples com tabela pequena. Depois da carga quase não há escrita

Carga com COPY	INSERT linha a linha	Faz diferença grande no volume que carregamos
Agregar antes de carregar	Guardar pessoa por pessoa	O limite é 1 GB, e agregar resolve a LGPD junto
Terraform	Deixar só no painel	O enunciado pede, e deixa o provisionamento auditável

Em todas essas escolhas a gente partiu do tamanho real do dado. O setor de TI é 1,23% do emprego formal, então o que sobra depois do filtro é pequeno o bastante pra ferramenta simples dar conta. Usar PySpark aqui ia dar bem mais trabalho pra resolver o mesmo problema.